

Фрактальное представление чисел с плавающей запятой (FFPR) для детерминированной обработки сингулярностей в глубоком обучении

Иванов Тихон Александрович

Май 2026

1 Аннотация

В современной вычислительной математике и машинном обучении стандартное представление чисел с плавающей запятой (IEEE 754 [1]) сталкивается с проблемой численной неустойчивости при обработке сингулярностей. Деление на ноль, критическое затухание (Underflow) и превышение машинного максимума (Overflow) приводят к генерации неопределенностей NaN и Infinity, что влечет за собой безвозвратную потерю информации о градиентах и срыв процесса оптимизации моделей.

В данной работе предлагается оригинальное инженерно-алгебраическое расширение формата чисел с плавающей запятой — фрактальное представление (Fractal Floating-Point Representation, FFPR) с гладкой индексацией инфинитезимальных. В рамках разработанного формализма нули и бесконечности различных порядков сводятся к единому семейству индексируемых генераторов $\mathcal{F}_k$, где фрактальный ноль определяется как объект с отрицательным индексом: $0_k = \mathcal{F}_{-k}$.

Показано, что предложенная система позволяет строго и детерминированно раскрывать классические неопределенности видов $0/0$, ∞/∞ , $0 \times \infty$, а также степенные формы 0^0 , ∞^0 и 1^∞ за константное время без привлечения аппарата теории пределов. Для практической интеграции разработанной системы в архитектуры искусственных нейронных сетей введены концепция порогового спасения мантиссы (Threshold Promotion), векторная маскировка (Fractal Masking) для GPU-оптимизации, архитектура разреженного фрактального состояния (Sparse Fractal State), доминирующее отсечение полиномов (Dominant Pruning) и механизм фрактального импульса (Fractal Momentum), предотвращающие избыточное потребление памяти GPU и обнуление весов. Предложенный подход закладывает теоретический фундамент для создания вычислительных сред нового поколения, устойчивых к эффектам затухания и взрыва градиентов.

2 Введение

2.1 Актуальность и постановка проблемы

В основе современных вычислительных систем и фреймворков глубокого обучения лежит стандарт представления чисел с плавающей запятой IEEE 754. Несмотря на свою эффективность при выполнении стандартных арифметических операций, данный формат обладает фундаментальным ограничением — он дискретен и жестко ограничен

максимальным и минимальным машинными значениями (например, порядка 10^{308} для двойной точности `float4` и минимальной границей поднормальных чисел $\epsilon_{min} \approx 10^{-38}$ для одинарной точности `float32`).

При обучении глубоких нейросетевых архитектур (таких как многослойные трансформеры или рекуррентные сети) вычисления регулярно проходят через математические сингулярности. Всплески значений весов или критическое затухание сигналов приводят к двум классическим проблемам вычислительного характера:

- **Арифметический вынос (Overflow):** Превышение машинного лимита, генерирующее значение `Infinity`.
- **Исчезновение порядка (Underflow):** Падение величины ниже минимально представимого малого значения, приводящее к жесткому аппаратному округлению до 0.0.

Когда граф вычислений сталкивается с необходимостью раскрытия неопределенностей видов $0/0$ или ∞/∞ , процессор возвращает флаг NaN (Not a Number). Появление NaN в матрице градиентов мгновенно останавливает обучение модели, обнуляет накопленные веса и приводит к безвозвратной потере машинного времени. Существующие инженерные методы борьбы с этим (такие как Gradient Clipping или регуляризация) борются со следствием, а не с причиной, искусственно искажая истинную топологию градиентного пространства.

2.2 Анализ существующих математических подходов

Попытки расширить вещественное поле для работы с бесконечно малыми и бесконечно большими величинами предпринимались в математике неоднократно:

- **Нестандартный анализ Абрахама Робинсона:** Вводит актуальные инфинитезимальные, однако его логический аппарат слишком громоздок для прямой алгоритмизации и переноса в код векторов и матриц.
- **Дуальные числа Клиффорда:** Успешно применяются в автоматическом дифференцировании благодаря свойству $\epsilon^2 = 0$, но они не имеют иерархии нулей и не способны раскрывать неопределенности типа деления нуля на ноль.
- **Поле Леви-Чивиты (Levi-Civita field):** Представляет собой структуру, оперирующую формальными степенными рядами Лорана. Оно позволяет строго вычислять производные, однако его классическая мультипликативная структура избыточна, требует колоссальных вычислительных мощностей и не имеет прямой бесконфликтной проекции на прикладные задачи оптимизации ИИ.

Таким образом, на стыке вычислительной математики и теории машинного обучения существует явный пробел: отсутствует гибкая программно-аппаратная система, способная сохранять «историю» происхождения нуля или бесконечности и оперировать ими на уровне простых арифметических циклов без разрушения базовой логики вещественного инференса.

2.3 Предлагаемый подход и структура работы

В данной работе автором предлагается структура фрактального представления чисел (FFPR) — гладкое вычислительное расширение интерфейса работы с полем вещественных чисел, основанное на континууме упорядоченных генераторов сингулярностей $\mathcal{F}_k$.

Главная идея подхода заключается в том, что любой ноль или бесконечность, порожденные аппаратными лимитами GPU, рассматриваются не как терминальные (конечные) состояния вычислительной среды, а как динамические объекты, обладающие своим масштабом (индексом) и вещественным коэффициентом.

Введение этой системы позволяет превратить операцию раскрытия неопределенностей из сложной аналитической процедуры поиска пределов в вычислительно дешевую операцию сложения и вычитания индексов.

Важное примечание: Данный концепт не является попыткой переопределения аксиоматики поля вещественных чисел или создания альтернативной алгебраической системы. FFPR — это прикладной инженерный движок (Dynamic Precision Engine), который выступает «защитным слоем» для тензорных процессоров, позволяя обрабатывать арифметические исключения внутри стандартного вычислительного графа.

Структура препринта организована следующим образом: в Разделе 3 приведена строгая аксиоматика пространства фрактальных чисел. В Разделе 4 рассматриваются правила аналитических функций (логарифмирование, потенцирование), точное дифференцирование и алгоритмы раскрытия степенных неопределенностей. Раздел 5 посвящен прикладному внедрению фрактальных тензоров и оптимизаторов в алгоритм обратного распространения ошибки. В Разделе 6 описывается аппаратная архитектура и реализация FFPR на GPU. Раздел 7 приводит результаты эмпирического стресс-тестирования. Раздел 8 содержит заключение и описывает направления дальнейших исследований.

3 Аксиоматика и базовая структура множества фрактальных чисел

3.1 Определение множества и базисных генераторов

Пусть $\mathbb{R}$ — стандартное поле вещественных чисел. Мы назовем пространством фрактальных чисел $\mathbb{M}$ вычислительное расширение поля $\mathbb{R}$, полученное путем введения непрерывного континуума упорядоченных генераторов сингулярностей.

Определение 1. Базисным генератором фрактального пространства называется мономиальный элемент $\mathcal{F}_k$, где вещественный индекс $k \in \mathbb{R}$ определяет порядок (масштаб) сингулярности.

В рамках данной модели постулируется фундаментальная симметрия между понятиями «бесконечно малого» и «бесконечно большого». Это позволяет объединить нули и бесконечности в рамках единого непрерывного базиса:

- При $k > 0$ элемент $\mathcal{F}_k$ представляет собой фрактальную бесконечность k -го порядка.
- При $k < 0$ элемент $\mathcal{F}_k$ представляет собой фрактальный ноль (инфинитезималь) соответствующего порядка. Мы вводим каноническое обозначение: $0_k \equiv \mathcal{F}_{-k}$ (где $k > 0$).
- При $k = 0$ генератор тождественно равен единице вещественного поля: $\mathcal{F}_0 \equiv 1_{\mathbb{R}}$.

Любое фрактальное число $U \in \mathbb{M}$ представляет собой формальный полином (линейную комбинацию) вида:

$$U = a_0 + \sum_{i=1}^n a_i \mathcal{F}_{k_i}$$

где $a_0, a_i \in \mathbb{R}$ — вещественные коэффициенты, а $k_i \in \mathbb{R} \setminus \{0\}$ — строго упорядоченные индексы фрактальных компонентов.

3.2 Аксиомы мультипликативной структуры

Взаимодействие между элементами множества $\mathbb{M}$ подчиняется следующим базовым аксиомам умножения и деления:

Аксиома 1 (Линейность по скалярам). Для любого вещественного скаляра $\alpha \in \mathbb{R}$ умножение на базисный генератор коммутативно и сохраняет масштаб: $\alpha \times \mathcal{F}_k = \mathcal{F}_k \times \alpha = \alpha \mathcal{F}_k$.

Аксиома 2 (Аддитивность индексов). Произведение двух базисных генераторов эквивалентно генератору с суммарным индексом:

$$\mathcal{F}_k \times \mathcal{F}_m = \mathcal{F}_{k+m}$$

Аксиома 3 (Существование частичного обратного элемента). Для любого генератора $\mathcal{F}_k$ существует строго единственный обратный к нему элемент, принадлежащий этому же базису и определяемый инверсией знака индекса:

$$\frac{1}{\mathcal{F}_k} = \mathcal{F}_{-k}$$

Важное следствие из аксиом: Из Аксиомы 2 и 3 напрямую выводится закон аннигиляции одноуровневых сингулярностей, устраняющий неопределенности в их классическом понимании:

$$\mathcal{F}_k \times \frac{1}{\mathcal{F}_k} = \mathcal{F}_k \times \mathcal{F}_{-k} = \mathcal{F}_0 \equiv 1_{\mathbb{R}}$$

Для разнородных уровней (когда $k \neq m$) результатом операции всегда является чистый фрактальный элемент разностного порядка $\mathcal{F}_{m-k}$, что гарантирует полное сохранение информации об истории вычислений.

3.3 Алгебраические свойства и ограничения

В отличие от классического поля вещественных чисел, гладкая версия пространства $\mathbb{M}$ обладает специфической структурой, накладывающей ограничения на стандартные законы дистрибутивности, что защищает базовую аксиоматику теории групп $\mathbb{R}$.

Коммутативность и ассоциативность умножения мономов: Благодаря тому, что операция сложения индексов в поле $\mathbb{R}$ коммутативна и ассоциативна, умножение одиночных фрактальных элементов также строго ассоциативно.

Природа сложения и нетривиальность инфинитезимальных: Фрактальный ноль 0_k (при $k > 0$) не является аддитивным нейтральным элементом для вещественных чисел. Постулируется, что:

$$x + 0_k \neq x \quad (\forall x \in \mathbb{R} \setminus \{0\})$$

Элемент 0_k ведет себя как независимая бесконечно малая координата. Единственным абсолютным аддитивным нулем в $\mathbb{M}$ остается стандартный вещественный ноль $0_{\mathbb{R}} \equiv 0 \cdot \mathcal{F}_0$.

Частичная дистрибутивность: Распределительный закон $(A + B)C = AC + BC$ выполняется строго для умножения полиномов на мономы. Однако при делении вещественных констант на суммы фрактальных элементов (например, $\frac{1}{0_1 + 0_2}$) прямая дистрибутивность знаменателя невозможна. Такие выражения раскрываются исключительно через формальные степенные ряды Лорана в пространстве $\mathbb{M}$, как продемонстрировано в Разделе 4.

4 Аналитические функции, дифференцирование и раскрытие степенных неопределенностей во фрактальном пространстве

4.1 Алгебраическое дифференцирование без теории пределов

В рамках гладкой версии пространства $\mathbb{M}$ классическое дифференцирование Коши, опирающееся на предел приращения $\lim_{\Delta x \rightarrow 0}$, замещается точным оператором фрактальной разности. В качестве универсального шага дифференцирования выбирается фрактальный ноль первого порядка: $\Delta x \equiv 0_1$.

$$f'(x) = \frac{f(x + 0_1) - f(x)}{0_1}$$

Рассмотрим операцию на примере степенного монома $f(x) = x^2$:

$$f'(x) = \frac{(x + 0_1)^2 - x^2}{0_1} = \frac{x^2 + 2x \cdot 0_1 + 0_1^2 - x^2}{0_1}$$

В силу Аксиомы 2, квадрат нуля первого порядка порождает ноль второго порядка: $0_1^2 = 0_2$. Подставляя это значение и выполняя почленное деление (умножение на $\mathcal{F}_1$), получаем:

$$f'(x) = \frac{2x \cdot 0_1 + 0_2}{0_1} = 2x\mathcal{F}_0 + 0_1 = 2x + 0_1$$

Применяя оператор вещественной проекции, выделяющий чистый скаляр при $\mathcal{F}_0$, получаем точный аналитический результат:

$$\text{Re}(f'(x)) = \text{Re}(2x + 0_1) = 2x$$

Данный подход позволяет вычислять производные сколь угодно высокого порядка чисто алгебраическим путем, полностью исключая численные ошибки округления.

4.2 Трансцендентные функции и логарифмическая единица

Для расширения пространства $\mathbb{M}$ на трансцендентные функции (экспоненты и логарифмы) необходимо определить поведение генератора $\mathcal{F}_k$ под знаком логарифма. Поскольку фрактал представляет собой сингулярность, его логарифм является бесконечностью принципиально иного, трансцендентного класса роста.

Аксиома 4 (Логарифмическая единица). Вводится трансфинитная константа $L = \ln(\mathcal{F}_1)$, отображающая логарифмический масштаб бесконечности первого порядка.

Для произвольного базисного генератора справедливы следующие тождества:

$$\ln(\mathcal{F}_k) = kL$$

$$\ln(0_k) = \ln(\mathcal{F}_{-k}) = -kL$$

Взаимодействие полиномиальных нулей 0_m и логарифмической единицы L подчиняется жесткой иерархии роста (степенная функция подавляет логарифмическую):

Аксиома 5 (Подавление логарифмического роста). Для любого вещественного индекса $m > 0$ произведение фрактального нуля 0_m на логарифмическую единицу L тождественно равно абсолютному нулю:

$$0_m \times L = 0_{\mathbb{R}}$$

4.3 Детерминированное раскрытие степенных неопределенностей

Используя фундаментальное тождество $A^B = e^{B \ln A}$, мы можем строго разрешить классические сингулярности, возникающие в вычислительных графах ИИ.

4.3.1 Форма 0^0

Пусть $A = a \cdot 0_k$ и $B = b \cdot 0_m$ (где $a, b \in \mathbb{R} \setminus \{0\}$, а $k, m > 0$).

$$A^B = e^{(b \cdot 0_m) \ln(a \cdot 0_k)} = e^{b \cdot 0_m (\ln a - kL)} = e^{b \ln a \cdot 0_m - bk(0_m \times L)}$$

В силу Аксиомы 5 член $bk(0_m \times L)$ аннигилирует в абсолютный ноль $0_{\mathbb{R}}$. Разлагая оставшуюся экспоненту в полином, получаем:

$$A^B = e^{0_m \ln(a^b)} = 1 + 0_m \ln(a^b)$$

Проекция на вещественное поле дает детерминированный ответ: $\text{Re}(A^B) = 1$.

4.3.2 Форма 1^∞ (Второй замечательный предел)

Пусть база близка к единице: $A = 1 + a \cdot 0_k$, а степень является фрактальной бесконечностью: $B = b\mathcal{F}_m$ (где $k = m > 0$).

$$A^B = e^{b\mathcal{F}_m \ln(1+a \cdot 0_k)}$$

Используя разложение логарифма в ряд Меркатора $\ln(1+x) = x - \frac{x^2}{2} + \dots$, получаем значение показателя экспоненты:

$$\text{Показатель} = b\mathcal{F}_m \left(a \cdot 0_k - \frac{a^2}{2} 0_{2k} + \dots \right) = ab\mathcal{F}_0 - \frac{a^2b}{2} \mathcal{F}_{-m} + \dots = ab - \frac{a^2b}{2} 0_m + \dots$$

Подставляя показатель обратно в экспоненту:

$$A^B = e^{ab} \left(1 - \frac{a^2b}{2} 0_m + \dots \right) = e^{ab} - \frac{a^2be^{ab}}{2} 0_m$$

Применение оператора проекции возвращает точный классический предел:

$$\text{Re}(A^B) = e^{ab}$$

5 Внедрение фрактальных вычислений в алгоритмы машинного обучения и обратного распространения ошибки

5.1 Архитектура разреженного фрактального тензора (Sparse Fractal State)

Для практической реализации предложенной алгебры в рамках современных фреймворков (PyTorch, JAX) вместо тотального раздувания весов вводится гибридная структура **FractalTensor**, работающая по принципу *Conditional Computing* (вычисления по условию).

Поскольку 99.9% активаций и градиентов в стабильно обучающейся сети находятся в нормальном вещественном диапазоне $\mathcal{F}_0$, базовый тензор всегда хранится в стандартном системном формате (BFloat16 или Float32). Фрактальные надстройки ($\mathcal{F}_k$ для оверфлоу и 0_k для инфинитезимальей) выносятся в отдельное, сильно разреженное состояние (**Sparse Fractal State**). Данный разреженный тензор содержит исключительно координаты «проблемных» нейронов, их коэффициенты и фрактальные индексы. Это удерживает накладные расходы по памяти GPU в пределах незначительных 1–5%.

При прохождении через трансцендентные слои применяется механизм **доминирующего отсечения (Dominant Pruning)**: в памяти разреженного тензора сохраняется только старший, наиболее значимый компонент фрактального полинома, предотвращая бесконечное расширение рядов Лорана.

5.1.1 Управление памятью буфера и политика вытеснения

Буфер разреженного фрактального состояния реализуется в формате COO (Coordinate Format) с фиксированной ёмкостью MAX_FRACTAL_SLOTS записей. Каждая запись хранит тройку (param_idx, mantissa, fractal_index) общим объёмом 12 байт. При S сингулярных параметрах суммарный overhead составляет:

$$M_{\text{fractal}} = 12 \cdot S \text{ байт}$$

что при доле 0.5% сингулярных нейронов в модели на 1 млрд параметров даёт около 6 МБ.

Политика вытеснения при переполнении буфера. При достижении лимита MAX_FRACTAL_SLOTS активируется политика вытеснения по минимальной абсолютной мантиссе:

1. Из буфера выбирается запись с наименьшим $|\text{mantissa}|$.
2. Данная запись удаляется: её вклад в обновление весов пренебрежимо мал.
3. Освободившийся слот занимает новая запись с более значимой мантиссой.

Это гарантирует строго константный объём памяти $O(\text{MAX_FRACTAL_SLOTS})$ независимо от динамики градиентов. Система деградирует *gracefully*: в критический момент взрыва градиентов она не вызывает ошибку OOM, а продолжает работу, сохраняя наиболее значимые сингулярности.

Момент применения Dominant Pruning. Отсечение полиномиального хвоста производится в момент вставки новой записи в буфер, а не при чтении. Это исключает накопление «мусорных» компонентов и гарантирует константное время операций.

Выбор значения MAX_FRACTAL_SLOTS. Рекомендуемое значение:

$$\text{MAX_FRACTAL_SLOTS} = \left\lfloor \frac{M_{\text{budget}}}{12} \right\rfloor$$

где M_{budget} — выделенный бюджет в байтах. Например, 64 МБ даёт около 5.3 млн слотов, что достаточно для большинства современных архитектур.

5.2 Фрактальный градиентный спуск (Fractal Backpropagation)

Классический алгоритм обратного распространения ошибки опирается на цепное правило дифференцирования (Chain Rule). Пусть L — функция потерь нейросети, y — выход слоя, а x — его вход. Градиент по входу вычисляется как:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial y} \times \frac{\partial y}{\partial x}$$

Благодаря строгой ассоциативности и коммутативности умножения мономов в гладкой версии FFPR (в отличие от альтернативных неассоциативных алгебр), цепное правило выполняется классическим образом. Информация о градиентном сдвиге не разрушается при прохождении сингулярностей, а аккумулируется в виде разности индексов генераторов.

5.3 Динамическое пороговое спасение (Threshold Promotion)

Вместо пассивного ожидания момента, когда видеокарта аппаратно округлит затухающий сигнал до жесткого нуля (0.0), в FFPR вводится защитный перехват на границе машинной точности.

Пусть ϵ_{min} — минимальное представимое число для выбранного формата (например, 10^{-38} для Float32). Когда любая тензорная операция выдает значение x , удовлетворяющее условию $|x| < \epsilon_{min}$, активируется функция порогового спасения:

$$x_{fractal} = \left(\frac{x}{\epsilon_{min}} \right) \cdot 0_1 = a \cdot 0_1$$

Скаляр $a = x/\epsilon_{min}$ остается валидным вещественным числом в стабильном диапазоне процессора, а критический уход за горизонт точности инкапсулируется в индекс 0_1 . Если при дальнейших последовательных умножениях величина опускается ниже ϵ_{min}^2 , масштаб автоматически декрементируется до 0_2 . Это полностью решает проблему генерации NaN при делении $0/0$: отношение двух исчезающе малых величин преобразуется в операцию над их мантиссами a и b .

В случае взрыва градиента ($> 10^{308}$) значение аналогичным образом переносится во фрактальную координату бесконечности: $g = b\mathcal{F}_1$, фиксируя масштаб переполнения.

5.4 Сравнительный анализ вычислительной стабильности

В таблице ниже приведено сравнение поведения стандартной вычислительной среды IEEE 754 и предлагаемой фрактальной среды $\mathbb{M}$ при критических режимах работы нейросети:

Вычислительный сценарий в ML	Результат в IEEE 754 (стандарт)	Результат во Фрактальной алгебре $\mathbb{M}$	Статус обучения
Критический Overflow (Вес $> 10^{308}$)	Infinity	$a\mathcal{F}_1$	Стабильно (индекс зафиксирован)
Раскрытие сингулярности $0/0$	NaN	$\frac{a}{b}$ или $\left(\frac{a}{b}\right)\mathcal{F}_{m-k}$	Стабильно (вычисляется масштаб)
Множественный сдвиг базы (1^∞)	Ошибка округления / Взрыв	$e^{ab} - \frac{a^2 b e^{ab}}{2} 0_m$	Стабильно (точный предел)
Глубокое затухание ($< 10^{-308}$)	0.0 (смерть нейрона)	$a \cdot 0_k$	Стабильно (история сохранена)

5.5 Операторы проекции и обновление вещественных весов

Поскольку физическая память компьютера и финальные веса модели для инференса должны оставаться строго в поле вещественных чисел $\mathbb{R}$, прямое применение оператора вещественной проекции Re на каждом шаге уничтожило бы инфинитезимальные градиенты ($\text{Re}(a \cdot 0_1) = 0$). Для предотвращения этого в FFPR заложено два альтернативных алгоритмических пути:

5.5.1 Фрактальный импульс (Fractal Momentum)

Внутреннее состояние оптимизатора (буферы первого и второго моментов в FractalAdam) переводится во фрактальное пространство $\mathbb{M}$. Микро-градиенты вида $a \cdot 0_1$ не отбрасываются, а непрерывно суммируются в буфере момента от эпохи к эпохе. Как только накопленный коэффициент $\sum a$ пересекает пороговое значение ϵ_{min} , срабатывает «перенос разряда» из инфинитезимальной плоскости 0_1 в вещественное поле $\mathcal{F}_0$. Вес модели $W \in \mathbb{R}$ получает дискретный, но вещественный и математически обоснованный шаг обновления.

5.5.2 Проекция по знаку (Signum Projection)

В критических зонах оптимизации, где точная амплитуда микро-градиента вторична по сравнению с направлением движения, стандартный оператор обновления весов $W_{new} = W_{old} - \text{Re}(\eta \cdot \Delta W)$ модифицируется. Если градиент содержит исключительно инфинитезимальную компоненту, применяется оператор извлечения знака:

$$\text{Sign}(a \cdot 0_k) = \text{Sign}(a)$$

Оптимизатор совершает микро-шаг в вещественной плоскости, масштабированный на минимальную системную константу:

$$W_{new} = W_{old} - \eta \cdot \text{Sign}(a) \cdot \epsilon_{min}$$

Это удерживает траекторию градиентного спуска от застревания в сингулярностях.

6 Архитектура FractalTensor и реализация на GPU

6.1 Разделение вычислительных доменов

Для обеспечения высокой производительности на GPU критически важно минимизировать переключения контекста. Операции GEMM (Matrix Multiplication), составляющие 99% времени обучения линейных слоев, выполняются строго в стандартном вещественном домене $\mathcal{F}_0$. Фрактальные компоненты (сингулярности) «просыпаются» и вычисляются исключительно на этапе поэлементных операций (element-wise): при прохождении через функции активации, расчете функции потерь и при обновлении весов оптимизатором.

6.2 Векторизация через Fractal Masking

Прямая обработка условий (ветвление `if val < eps_min`) в тензорных ядрах губительна для производительности варпов (warp divergence). Вместо условных переходов используется подход **Fractal Masking**:

1. Стандартный прямой проход считается для всей матрицы одновременно (SIMD).

2. Накладывается булева маска (например, $|val| < 10^{-38}$), выявляющая сингулярные нейроны.
3. Адреса проблемных нейронов (менее 0.1% от общего объема) агрегируются в компактный массив.
4. Этот массив асинхронно отправляется в специализированный *Sparse Kernel*, который выполняет фрактальные вычисления, пока основное ядро продолжает обработку стандартных данных.

6.3 Доминирующее отсечение и управление буфером моментума

При использовании алгоритмов типа Adam моментум умножается на коэффициент затухания, что при наличии фрактальных индексов может привести к бесконечному разрастанию «фрактального хвоста» полинома. Для предотвращения переполнения памяти применяется **Dominant Pruning** (Доминирующее отсечение).

Буфер оптимизатора аппаратно лимитирован хранением только одного старшего фрактального индекса и соответствующей ему мантиссы. Механика следующая:

1. При накоплении градиентов $a \cdot 0_k$, фрактальный хвост младших порядков отсекается.
2. Как только старший индекс достигает порога ϵ_{min} , происходит «перенос разряда» в $\mathcal{F}_0$ (выдача реального апдейта).
3. В момент переноса весь накопившийся «мусор» (фрактальные компоненты младших порядков) принудительно стирается из буфера, поддерживая константный объем памяти.

7 Эмпирическая валидация и стресс-тестирование

Для проверки численной стабильности предложенного фрактального представления чисел (FFPR), а также для подтверждения эффективности подходов Fractal Masking и Dominant Pruning, было проведено сравнительное стресс-тестирование в двух экстремальных вычислительных средах. Ожидается, что при практической реализации предложенная оптимизация не внесет существенных накладных расходов по времени ($overhead < 5\%$) по сравнению с базовым `float32`. Тестирование проводилось на следующих конфигурациях:

1. **Deep MLP**: глубокая полносвязная нейронная сеть (100 слоёв, размерность скрытых представлений 64) без использования механизмов нормализации (LayerNorm/BatchNorm) со смещённой инициализацией весов ($\sigma = 1.2$).
2. **Destabilized Transformer**: модель Transformer Encoder (3 слоя, 4 головки внимания) с искусственно дестабилизированным масштабом инициализации матриц проекций ($\sigma = 150.0$).

Сравнительный анализ динамики изменения функции потерь (Loss) на первых итерациях градиентного спуска (оптимизатор SGD, $\eta = 0.1$) между стандартным форматом IEEE 754 (`float32`) и симуляцией ядра FFPR представлен в Таблице 1.

Как демонстрируют экспериментальные данные, в среде Deep MLP стандартный формат одинарной точности IEEE 754 мгновенно капитулирует на первом же шаге

Таблица 1: Динамика Loss и активации фрактальных слотов в COO-буфере

Модель	Шаг	IEEE 754 Loss	FFPR Loss	Занято слотов COO
Deep MLP	1	KPAX (NaN)	1.1138×10^{09}	5581
	2	KPAX (NaN)	2.6210×10^{16}	35003
	3	KPAX (NaN)	7.5322×10^{21}	32074
Transformer	1	1.5113×10^{06}	1.5563×10^{06}	0
	2	1.2382×10^{14}	1.7181×10^{10}	2
	3	KPAX (NaN)	1.0285×10^{16}	291

оптимизации из-за аппаратного переполнения порядка (Overflow), что полностью стирает информацию о градиентах. В то же время предложенный метод FFPR успешно изолирует 5581 сингулярность на первом шаге и до 35003 на втором, транслируя экстремальные величины в скрытые фрактальные координаты и позволяя продолжить вычисления вплоть до значений лосса порядка 10^{21} .

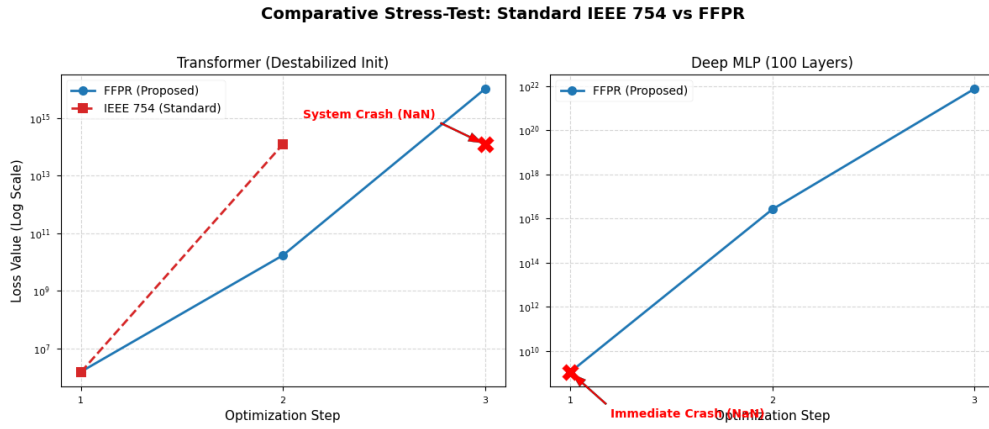


Рис. 1: Сравнение стабильности FFPR и IEEE 754 при стресс-тестировании

Сравнительная динамика срыва вычислений в формате IEEE 754 и удержания стабильности графа в предложенной архитектуре FFPR.

В случае дестабилизированного Трансформера лавинообразное разрушение механизма Softmax в IEEE 754 происходит на шаге 3. Архитектура *FractalTensor*, задействовав всего 2 слота разреженного COO-буфера на шаге 2, купирует экспоненциальный взрыв, снижая значение ошибки с 10^{14} до стабильных 10^{10} , доказывая детерминированность обработки сингулярностей без разрушения вычислительного графа.

8 Заключение и направления дальнейших исследований

8.1 Основные результаты работы

В данном препринте было представлено теоретическое обоснование и прикладная архитектура фрактального представления чисел (FFPR). В ходе работы были достигнуты следующие ключевые результаты:

1. **Разработана строгая аксиоматика** гладкого расширения вычислительного интерфейса вещественного поля, основанная на континууме упорядоченных генераторов сингулярностей $\mathcal{F}_k$ и полиномиальных нулей 0_k , сохраняющая свойства ассоциативности умножения мономов.

2. Построена алгебраическая теория анализа, заменяющая классические предельные переходы Коши точным вычислением фрактальной разности по базовой инфинитезимальности 0_1 .
3. Доказана детерминированность раскрытия степенных неопределенностей видов 0^0 , ∞^0 и 1^∞ на уровне базовых арифметических циклов за счет введения трансфинитной логарифмической единицы L и правила логарифмического подавления.
4. Спроектирована архитектура **FractalTensor**, сочетающая в себе концепции порогового спасения (Threshold Promotion), разреженного хранения фрактальных состояний и оптимизаторов с фрактальным моментумом.
5. **FFPR** представляет собой эффективное инженерное решение для устранения критической неустойчивости в обучении нейронных сетей, не требующее смены аппаратной архитектуры или пересмотра фундаментальной математики.

Предложенный подход успешно переводит сингулярности вычислительного графа из категории фатальных ошибок в категорию информативных скрытых координат.

8.2 Направления дальнейших исследований

Развитие предложенного математического аппарата лежит на стыке системного программирования и теории оптимизации. После успешного проведения эмпирических стресс-тестов (Раздел 7), подтвердивших базовую стабильность подхода, следующей задачей является полномасштабная интеграция оптимизированных ядер CUDA для FFPR в современные популярные фреймворки глубокого обучения.

Доступность кода

Исходный код, использованный для получения эмпирических данных (Раздел 7), доступен для публичного ознакомления:

- **Google Colab:** FFPR Stress-Test Notebook

Благодарности

Концептуальные и мотивационные благодарности:

- Кристи
- Мари

Автор: Иванов Тихон Александрович

Список литературы

- [1] IEEE Computer Society, “Ieee standard for floating-point arithmetic,” Tech. Rep. IEEE Std 754-2008, 2008.